



Universidade do Minho
Escola de Engenharia

DomiUM

Programação Orientada aos Objetos

Alexandre Gabriel Silva Machado (A106801)
Daniel Ramos de Almeida Marcos Costa (A108708)
Simão Pedro Ribeiro Santos (A109767)

10 de maio de 2026
Braga, Portugal

Índice de conteúdos

1. Introdução	3
2. Arquitetura Geral da Solução	4
2.1 Organização em pacotes	5
2.2 A classe DomiUM como fachada	6
2.3 Separação entre GestorCasas e GestorUtilizadores	7
3. Análise do Diagrama de Classes	8
3.1 Entidades núcleo do domínio	8
3.2 Hierarquia de Dispositivos	9
3.3 Padrão Command	11
3.4 Condições e ambiente interior	12
3.5 Automações, escalonamentos e cenários	13
3.6 Criação extensível de Dispositivos	15
3.7 Histórico e Sugestões automáticas	15
3.8 Relógio simulado, estatísticas e persistência	17
3.9 Hierarquia de exceções	18
4. Princípios de Programação Orientada a Objetos aplicados	20
4.1 Encapsulamento	20
4.2 Abstração	20
4.3 Herança e Polimorfismo	21
4.4 Composição	21
4.5 Padrões e técnicas de desenho utilizados	22
5. Funcionalidades da aplicação	23
6. Conclusão	25
7. Anexos	26

Índice de tabelas

Tabela 1: Turnaround time de 5 comandos idênticos (sleep 4) submetidos
concorrentemente.

Error! Bookmark not defined.

1. Introdução

O projeto `DomiUM` consiste numa aplicação de consola para simular a gestão de uma casa inteligente. A aplicação permite criar utilizadores, casas, divisões e dispositivos, associar dispositivos às divisões e atuar sobre eles individualmente ou através de mecanismos de automação.

Além da gestão base das entidades, foram implementadas funcionalidades de estatística, persistência em ficheiro binário, cenários, escalonamentos, automações e geração de sugestões automáticas com base no histórico de interações dos utilizadores. O sistema inclui ainda um modelo de permissões por casa, permitindo distinguir utilizadores administradores de utilizadores normais.

A solução foi desenvolvida seguindo o paradigma da Programação Orientada aos Objetos, com separação entre domínio, controlo e interface textual. Foram aplicados princípios como encapsulamento, abstração, composição, herança e polimorfismo, bem como padrões de desenho como `Facade`, `Command`, `Factory`, e `Registry`.

2. Arquitetura Geral da Solução

A aplicação `DomiUM` foi organizada seguindo uma separação clara entre interface, controlo e domínio. Esta organização segue o padrão MVC. A vista trata apenas da interação textual, os controllers coordenam os menus e o domínio concentra as regras de negócio.

No contexto da arquitetura MVC, a camada de domínio corresponde ao `Model` da aplicação. Esta camada está concentrada no pacote `domus.domain` e contém a fachada `DomiUM`, as entidades principais, os gestores internos e as regras de negócio.

A classe `ConsoleView` representa a vista da aplicação. Esta classe não altera o estado do sistema nem contém regras de negócio. A sua responsabilidade é mostrar menus, ler dados introduzidos pelo utilizador e apresentar mensagens de sucesso ou erro.

A camada de controlo é composta pelo `DomiUMController` e pelos `controllers` especializados dos menus, como `DispositivosMenuController`, `CenariosMenuController`, `EscalonamentosMenuController`, `AutomacoesMenuController`, `EstatisticasMenuController` e `SugestoesMenuController`. O `DomiUMController` gere o menu principal e delega as funcionalidades mais específicas nos `controllers` auxiliares. Desta forma, evita-se que uma única classe de controlo fique demasiado extensa e difícil de manter.

No centro do domínio encontra-se a classe `DomiUM`, que funciona como fachada da aplicação. Os `controllers` comunicam com o domínio através desta classe e não acedem diretamente aos gestores internos, às coleções de casas ou às coleções de utilizadores. Assim, toda a comunicação com o domínio passa por um único ponto de entrada.

Internamente, a `DomiUM` delega parte do trabalho em classes gestoras, nomeadamente o `GestorCasas` e o `GestorUtilizadores`. O `GestorCasas` concentra a gestão das casas, divisões, dispositivos, cenários, escalonamentos e automações. O `GestorUtilizadores` concentra a criação de utilizadores, permissões e histórico de interações. Esta divisão permite que a fachada não tenha de implementar diretamente todos os detalhes do armazenamento e manipulação das entidades.

A Figura 1 apresenta uma visão simplificada desta organização, mostrando a relação entre a interface textual, os controllers e a fachada principal do domínio. A figura mostra apenas a organização geral da arquitetura. As restantes classes do domínio são apresentadas nas secções seguintes e no diagrama completo em anexo.

No diagrama, as linhas contínuas indicam associações diretas, as setas a tracejado indicam dependências de utilização e os losangos preenchidos representam composição.

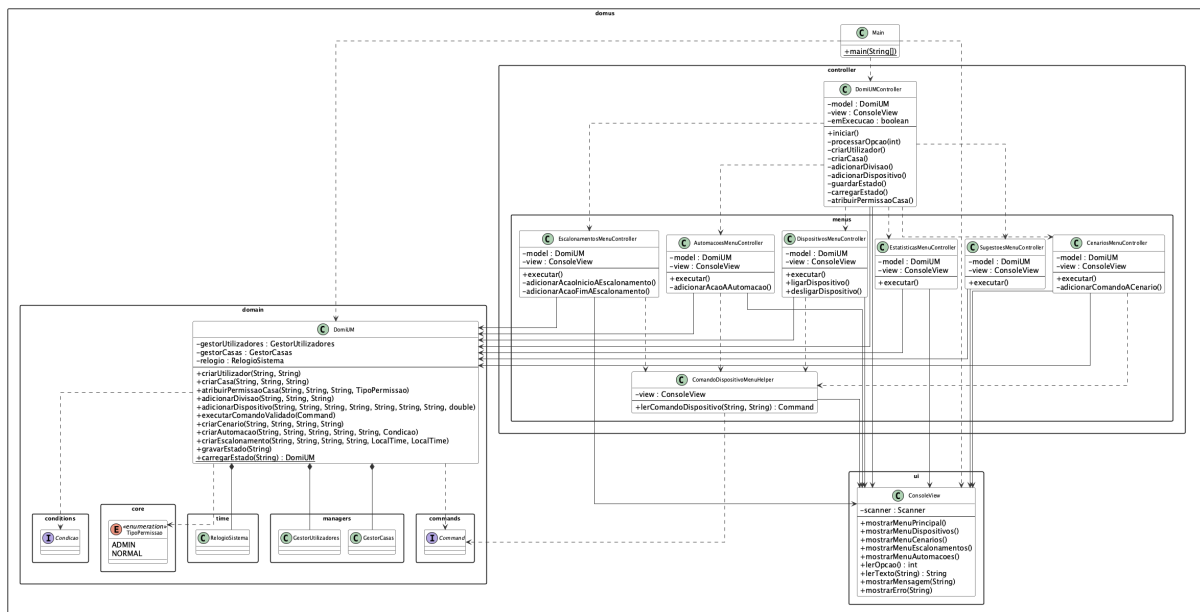


Figura 1. Visão simplificada da arquitetura MVC e da fachada de domínio.

2.1 Organização em pacotes

O projeto foi dividido em pacotes de acordo com a responsabilidade de cada conjunto de classes. Esta divisão torna a estrutura mais legível e ajuda a localizar rapidamente cada parte da aplicação.

Pacote	Responsabilidade
domus	Ponto de entrada da aplicação, através da classe Main.
domus.ui	Interface textual da aplicação, implementada em ConsoleView.
domus.controller	Controller principal da aplicação.
domus.controller.menus	Controllers dos submenus da aplicação de consola.
domus.domain	Fachada principal do domínio, representada pela classe DomiUM.
domus.domain.core	Entidades base, como Casa, Divisao, Utilizador e TipoPermissao.
domus.domain.devices	Hierarquia de dispositivos inteligentes.

<code>domus.domain.commands</code>	Comandos executáveis sobre dispositivos, seguindo o padrão Command.
<code>domus.domain.conditions</code>	Condições avaliáveis sobre o ambiente interior.
<code>domus.domain.automation</code>	Representação das automações.
<code>domus.domain.scheduling</code>	Representação dos escalonamentos temporais.
<code>domus.domain.scenarios</code>	Representação dos cenários.
<code>domus.domain.factories</code>	Criação extensível de dispositivos.
<code>domus.domain.managers</code>	Gestores internos de casas e utilizadores.
<code>domus.domain.environment</code>	Modelo do ambiente interior de uma divisão.
<code>domus.domain.history</code>	Registo das interações realizadas pelos utilizadores.
<code>domus.domain.suggestions</code>	Geração e aceitação de sugestões automáticas.
<code>domus.domain.statistics</code>	Objetos de resumo usados nas estatísticas.
<code>domus.domain.time</code>	Relógio simulado da aplicação.
<code>domus.domain.exceptions</code>	Hierarquia de exceções de domínio.

Tabela 1. Pacotes da aplicação e respetivas responsabilidades.

Esta organização foi pensada para acompanhar os conceitos principais do enunciado. As casas, divisões e utilizadores estão no núcleo do domínio. Os dispositivos têm uma hierarquia própria. As ações sobre dispositivos foram modeladas como comandos. As automações, escalonamentos e cenários reutilizam esses comandos, em vez de implementarem lógica própria para cada tipo de ação.

2.2 A classe DomiUM como fachada

A classe `DomiUM` é o ponto de entrada público para a camada de domínio. Isto significa que as operações realizadas a partir da interface passam por esta classe antes de chegarem às entidades internas.

Esta escolha evita que os `controllers` manipulem diretamente objetos como `Casa`, `Divisao`, `Utilizador` ou os mapas internos dos gestores. Por exemplo, quando um utilizador tenta adicionar um dispositivo a uma casa, o `controller` apenas recolhe os dados introduzidos no menu e chama um método da `DomiUM`. A fachada valida se o

utilizador existe, se a casa existe e se o utilizador tem permissão para administrar essa casa. Só depois delega a operação no gestor adequado.

A mesma lógica aplica-se a cenários, escalonamentos e automações. Mesmo quando o `controller` constrói um objeto `Command`, a validação da existência do dispositivo, das permissões e da compatibilidade entre comando e tipo de dispositivo continua a ser feita no `domínio`. Assim, a interface não precisa conhecer os detalhes internos dos dispositivos.

Se no futuro a forma como as casas ou permissões são guardadas mudar, os `controllers` não precisam ser alterados, desde que a interface pública da `DomiUM` se mantenha estável.

2.3 Separação entre GestorCasas e GestorUtilizadores

A classe `DomiUM` não guarda diretamente toda a lógica da aplicação. Em vez disso, utiliza gestores internos para separar responsabilidades. O `GestorCasas` é responsável pelas operações ligadas às casas e ao seu conteúdo. O `GestorUtilizadores` é responsável pelos utilizadores, permissões e histórico.

As regras relacionadas com casas ficam no gestor de casas e as regras relacionadas com utilizadores ficam no gestor de utilizadores. A fachada continua a coordenar as operações que envolvem ambos, como a criação de uma casa, em que é necessário criar a casa e atribuir permissão de administrador ao utilizador criador.

3. Análise do Diagrama de Classes

Depois de apresentada a arquitetura geral, importa analisar as principais partes do diagrama de classes. Em vez de apresentar todas as classes de uma só vez, esta secção divide o sistema em grupos temáticos. Esta abordagem facilita a leitura do diagrama e permite relacionar cada conjunto de classes com as funcionalidades que implementa.

O diagrama completo encontra-se no anexo, mas ao longo desta secção são apresentados excertos mais pequenos, focados nas zonas principais da aplicação. Estes excertos mantêm a mesma notação e o mesmo estilo visual do diagrama completo.

3.1 Entidades núcleo do domínio

As entidades principais do domínio encontram-se no pacote `domus.domain.core`. Este pacote contém as classes `Casa`, `Divisao`, `Utilizador` e o `enum TipoPermissao`.

A classe `Casa` representa uma instalação doméstica dentro da aplicação. Cada casa possui um identificador, um nome e várias coleções internas. Uma casa agrega divisões, cenários, automações e escalonamentos. Esta relação é uma relação de composição, pois estes elementos fazem parte do estado da casa dentro do sistema.

A classe `Divisao` representa uma divisão física de uma casa, como a sala, a cozinha ou o quarto. Cada divisão contém dispositivos e um objeto `AmbienteInterior`, onde são guardados valores como temperatura, humidade e luminosidade. Esta associação permite que as automações sejam avaliadas com base nas condições de uma divisão específica.

A classe `Utilizador` representa uma pessoa registada na aplicação. Cada utilizador tem um identificador, um nome, permissões associadas a casas e um histórico de interações. As permissões são guardadas por casa, o que permite que o mesmo utilizador seja administrador numa casa e utilizador normal noutra.

O `enum TipoPermissao` define os dois níveis de acesso existentes: `ADMIN` e `NORMAL`. Um utilizador com permissão `ADMIN` pode realizar operações de administração, como adicionar divisões, adicionar dispositivos ou atribuir permissões. Um utilizador com permissão `NORMAL` pode usar a casa, por exemplo executando comandos, cenários, automações ou escalonamentos, mas não pode alterar a sua estrutura.

Para guardar as coleções principais foram usados `Map`, normalmente com o identificador funcional como chave. Por exemplo, as casas são identificadas pelo seu id, os dispositivos pelo identificador e as divisões pelo nome. Esta escolha permite pesquisas diretas e evita percorrer listas sempre que se pretende encontrar uma entidade específica.

Nesta parte do domínio também se nota uma preocupação com encapsulamento. As coleções internas não são expostas diretamente. Em vez disso, as classes disponibilizam métodos de consulta ou iteradores sobre cópias protegidas. Isto evita que código externo consiga alterar o estado interno da casa, da divisão ou do utilizador sem passar pelas regras do domínio.

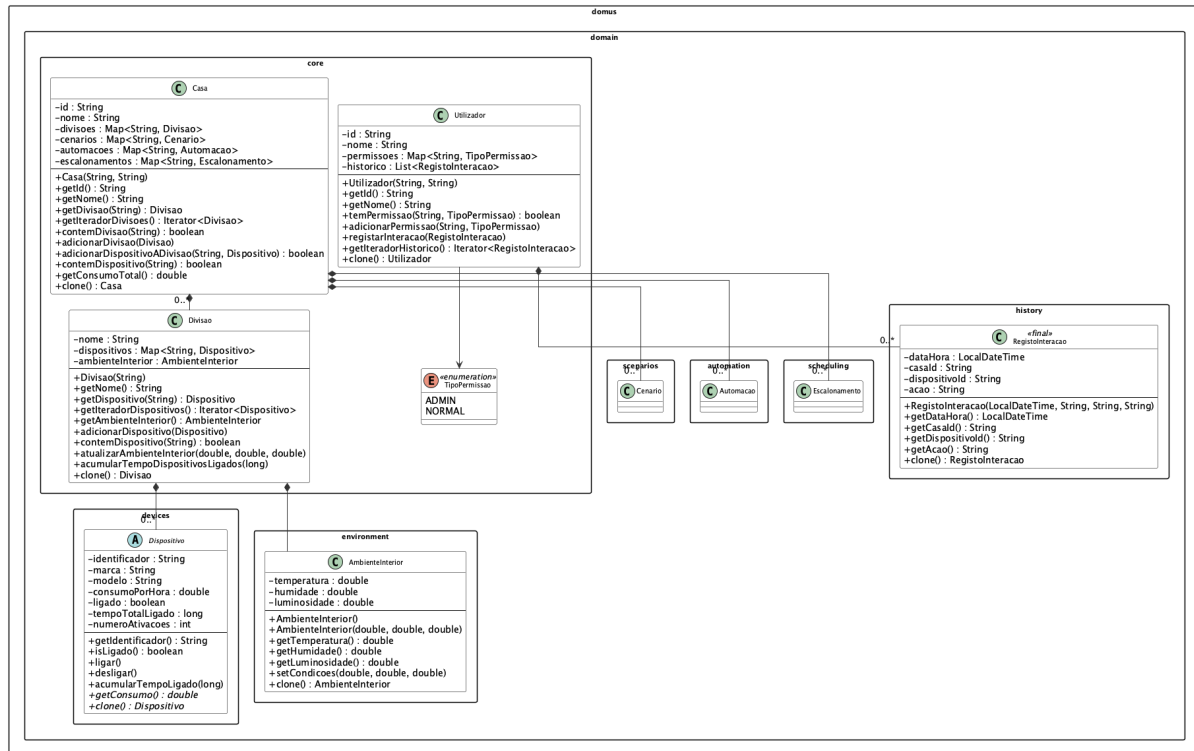


Figura 2. Entidades núcleo do domínio e respetivas relações.

3.2 Hierarquia de Dispositivos

Os dispositivos inteligentes encontram-se no pacote `domus.domain.devices`. A classe abstrata `Dispositivo` representa o comportamento comum a todos os dispositivos da aplicação. Nesta classe estão concentrados os atributos partilhados, como o identificador, a marca, o modelo, o consumo por hora, o estado ligado ou desligado, o tempo total ligado e o número de ativações.

Esta classe foi definida como abstrata porque não faz sentido criar um dispositivo genérico sem tipo concreto. Na aplicação existem vários dispositivos específicos, como lâmpadas, colunas, cortinas, ar condicionados, desumidificadores, fechaduras e portões de garagem. Todos partilham características comuns, mas cada um acrescenta estado e comportamento próprios.

A classe `LampadaInteligente`, por exemplo, acrescenta a intensidade e a temperatura de cor. A classe `ColunaInteligente` acrescenta o volume e a playlist atual. A classe

`CortinaInteligente` guarda a percentagem de abertura. O `ArCondicionadoInteligente` guarda a temperatura alvo e o modo de funcionamento. O `DesumidificadorInteligente` guarda a humidade alvo. A `FechaduraInteligente` guarda se está trancada, e o `PortaoGaragemInteligente` guarda se está aberto.

A decisão de usar herança nesta parte do projeto permite tratar todos os dispositivos de forma uniforme quando apenas interessam os dados comuns. Por exemplo, uma divisão pode guardar uma coleção de **Dispositivo**, sem precisar saber se cada elemento é uma lâmpada, uma coluna ou um portão. Quando é necessário comportamento específico, esse comportamento fica na subclasse correspondente.

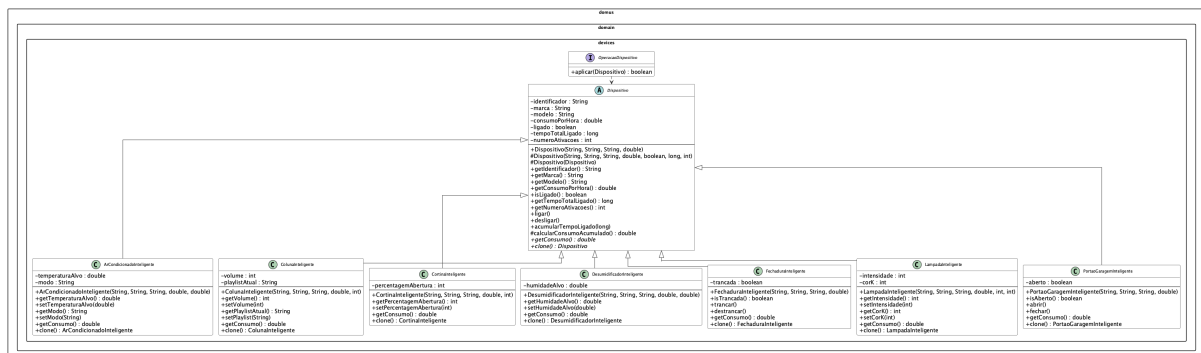
O método `getConsumo()` foi definido como abstrato para permitir que cada tipo de dispositivo possa ter a sua própria forma de calcular consumo. Na implementação atual, todas as subclasses reutilizam a lógica comum da superclasse, baseada no consumo por hora e no tempo total ligado. Mesmo assim, deixar este método abstrato torna a solução mais extensível. Se no futuro um dispositivo tiver uma fórmula de consumo diferente, basta alterar a sua subclasse.

A contabilização do tempo ligado foi centralizada na classe `Dispositivo`, através do método `acumularTempoLigado()`. Quando o relógio simulado avança, os dispositivos que estão ligados acumulam tempo de utilização.

Esta fórmula converte os minutos ligados em horas, uma vez que o consumo configurado nos dispositivos é dado por hora. Por exemplo, uma lâmpada com consumo de 10 Wh ligada durante 30 minutos terá um consumo estimado de 5 Wh.

A classe `Dispositivo` também define o método abstrato `clone()`. Cada subclasse é responsável por implementar a sua própria cópia, garantindo que o estado específico de cada dispositivo é preservado. Esta opção é importante para o encapsulamento, pois permite devolver cópias defensivas em vez de expor diretamente os objetos internos do domínio.

Outro elemento importante deste pacote é a interface `OperacaoDispositivo`. Esta interface representa uma operação genérica que pode ser aplicada a um dispositivo. A sua existência permite que algumas operações sejam passadas como comportamento, sem obrigar a fachada ou os gestores a conhecerem todos os detalhes de cada dispositivo. Esta ideia é usada em conjunto com os comandos, apresentados na secção seguinte.



3.3 Padrão Command

As ações sobre dispositivos foram modeladas através do padrão `Command`, implementado no pacote `domus.domain.commands`. Este padrão permite representar uma ação como um objeto. Em vez de a aplicação chamar diretamente métodos específicos dos dispositivos, como alterar o volume de uma coluna ou definir a intensidade de uma lâmpada, cria-se um objeto que representa essa intenção.

A interface `Command` define o contrato comum de todos os comandos. Cada comando sabe executar-se sobre a fachada `DomiUM`, através do método `execute(DomiUM)`, e sabe criar uma cópia de si próprio através do método `clone()`. Esta cópia é importante porque os comandos podem ser guardados dentro de cenários, automações e escalonamentos sem expor referências internas.

A classe abstrata `ComandoDispositivo` concentra os dados comuns aos comandos que atuam sobre dispositivos. Estes comandos guardam apenas o identificador do utilizador, o identificador da casa e o identificador do dispositivo. Esta decisão é importante para o encapsulamento e para a persistência. Em vez de guardar referências diretas para objetos do domínio, o comando guarda apenas os dados necessários para localizar o dispositivo no momento da execução.

Existem comandos concretos para as várias operações suportadas pela aplicação. Os comandos mais simples são `ComandoLigar` e `ComandoDesligar`. Além destes, existem comandos específicos de certos tipos de dispositivos, como `ComandoDefinirIntensidadeLampada`, `ComandoDefinirCorLampada`, `ComandoDefinirVolumeColuna`, `ComandoDefinirPlaylistColuna`, `ComandoDefinirAberturaCortina`, `ComandoDefinirTemperaturaArCondicionado`, `ComandoDefinirModoArCondicionado`, `ComandoDefinirHumidadeDesumidificador`, `ComandoAbrirPortao`, `ComandoFecharPortao`, `ComandoTrancarFechadura` e `ComandoDestrançarFechadura`.

Esta abordagem evita espalhar lógica específica dos dispositivos pelos `controllers` ou pelas classes que representam cenários, automações e escalonamentos. Por exemplo, um cenário não precisa saber se está a ligar uma lâmpada, a alterar o volume de uma coluna ou a fechar um portão. O cenário guarda uma lista de `Command` e executa cada comando pela mesma interface.

A mesma ideia é usada nos escalonamentos e nas automações. Um escalonamento guarda comandos para executar no início e no fim de um intervalo temporal. Uma automação guarda comandos para executar quando uma condição sobre o ambiente é satisfeita. Como todos estes mecanismos dependem da mesma interface `Command`, o sistema evita duplicação e torna-se mais fácil de estender.

A compatibilidade entre comandos e dispositivos é validada através do método `suportaDispositivo()`. Este método permite verificar, antes de guardar ou executar uma

ação agregada, se o comando faz sentido para o tipo de dispositivo indicado. Por exemplo, um comando para definir o volume de uma coluna não pode ser aplicado a uma lâmpada. Esta validação é feita no domínio, e não na interface, garantindo que as regras de negócio ficam centralizadas.

A alternativa seria implementar a lógica com vários `if`, `switch` ou `instanceof` espalhados pela aplicação. Essa solução seria mais difícil de manter, porque sempre que fosse criado um novo tipo de comando seria necessário alterar várias classes. Com o padrão Command, cada nova ação fica isolada numa nova classe de comando.

Também foi criado o `ComandoDispositivoMenuHelper`, na camada de controlo, para construir comandos a partir da consola. Este *helper* não valida regras de domínio e não manipula diretamente dispositivos. A sua responsabilidade é apenas recolher os dados necessários e instanciar o comando correto. Depois disso, a `DomiUM` continua responsável por validar permissões, existência do dispositivo e compatibilidade entre comando e dispositivo.

A Figura 4 mostra a hierarquia de comandos e a forma como os comandos concretos derivam de `ComandoDispositivo`.

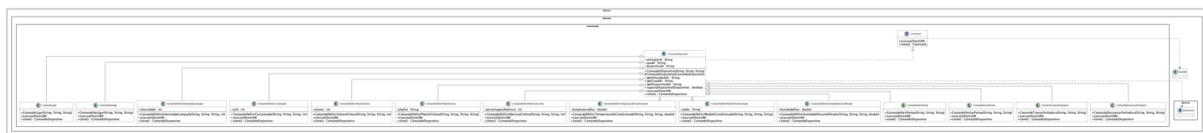


Figura 4. Hierarquia de comandos de dispositivo.

3.4 Condições e ambiente interior

As automações dependem da avaliação de condições sobre o estado do ambiente de uma divisão. Para representar esse ambiente foi criada a classe `AmbienteInterior`, no pacote `domus.domain.environment`. Esta classe guarda três valores principais: temperatura, humidade e luminosidade.

Cada `Divisao` contém um objeto `AmbienteInterior`. Isto permite associar as condições ambientais diretamente ao espaço físico onde os dispositivos estão instalados. Por exemplo, uma automação pode estar associada à sala e ser disparada quando a luminosidade dessa divisão fica abaixo de um determinado valor.

As condições foram modeladas através da interface `Condicao`, no pacote `domus.domain.conditions`. Esta interface define o método `avaliar(AmbienteInterior)`, que recebe o ambiente atual de uma divisão e devolve se a condição é verdadeira ou falsa.

Existem três implementações concretas desta interface: `CondicaoTemperatura`, `CondicaoHumidade` e `CondicaoLuminosidade`. Cada uma guarda um limite e um sentido

de comparação, permitindo representar condições como “temperatura maior que 25”, “humidade maior que 70” ou “luminosidade menor ou igual a 30”.

Cada condição encapsula uma forma diferente de avaliar o ambiente, mas todas obedecem à mesma interface. Assim, a classe Automacao não precisa saber se está a avaliar temperatura, humidade ou luminosidade. Apenas chama o método `avaliar()` da condição que recebeu.

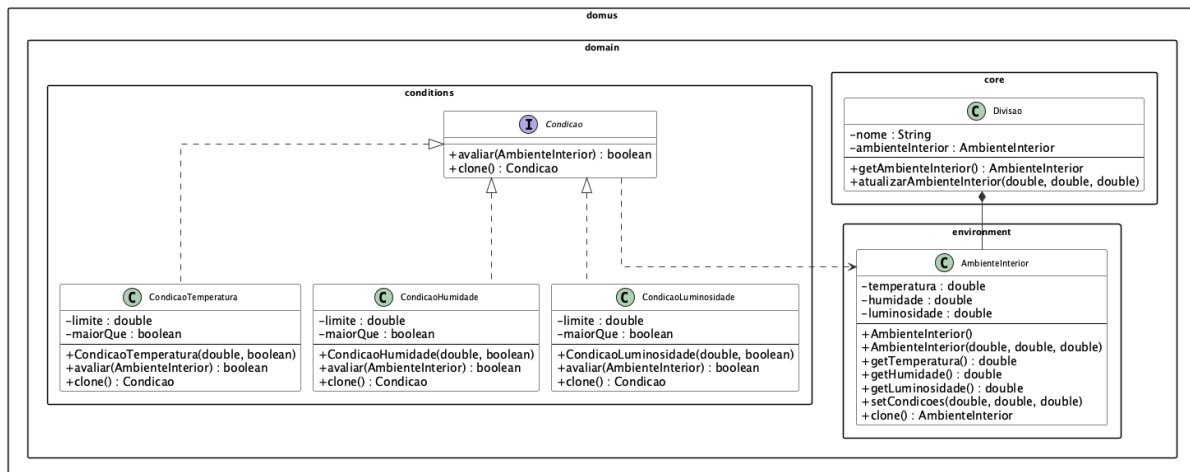


Figura 5. Condições avaliáveis sobre o ambiente interior.

3.5 Automações, escalonamentos e cenários

As funcionalidades de automações, escalonamentos e cenários representam formas diferentes de executar ações sobre dispositivos. Apesar de terem objetivos distintos, as três reutilizam a mesma abstração: uma lista de objetos `Command`.

A classe `Cenario`, no pacote `domus.domain.scenarios`, representa uma sequência de comandos definida pelo utilizador. Um cenário não depende de condições ambientais nem de tempo. Quando é executado, percorre a sua lista de comandos e executa cada um pela ordem em que foi adicionado. Isto permite representar situações como “Sair de casa”, “Jantar com amigos”, “Deitar” ou “Acordar”.

A classe `Automacao`, no pacote `domus.domain.automation`, associa uma divisão a uma condição e a uma lista de ações. Quando o ambiente de uma divisão é atualizado, a automação pode avaliar a sua condição sobre o `AmbienteInterior` dessa divisão. Se a condição for verdadeira, os comandos associados são executados. Desta forma, é possível representar regras como ligar uma lâmpada quando a luminosidade é baixa ou ligar um desumidificador quando a humidade ultrapassa um limite.

A classe `Escalonamento`, no pacote `domus.domain.scheduling`, representa uma regra temporal. Cada escalonamento possui uma hora de início, uma hora de fim, uma lista de

ações de início e uma lista de ações de fim. Quando o relógio simulado avança, a aplicação verifica se alguma dessas horas foi atingida e executa os comandos correspondentes.

A principal vantagem desta modelação é a reutilização. Cenários, automações e escalonamentos não precisam conhecer os tipos concretos de dispositivos nem os detalhes de cada operação. Todos guardam objetos **Command**, o que permite usar tanto comandos simples, como ligar e desligar, como comandos específicos, como definir intensidade de uma lâmpada, volume de uma coluna ou temperatura alvo de um ar condicionado.

Esta escolha também facilita a evolução da aplicação. Se for criado um novo comando, ele pode ser usado automaticamente nos três mecanismos, desde que seja construído pela interface de consola e validado pela **DomiUM**. Assim, a lógica de execução de ações não fica repetida em várias classes.

No caso das automações, a condição fica separada das ações. A automação apenas avalia a condição e, se esta for verdadeira, executa a lista de comandos. No caso dos escalonamentos, o relógio simulado é responsável por avançar o tempo, e a verificação temporal decide quando executar as ações. No caso dos cenários, a execução é manual e imediata.

A Figura 6 mostra como **Cenário**, **Automacao** e **Escalonamento** reutilizam a interface **Command**, embora sejam acionados por mecanismos diferentes.

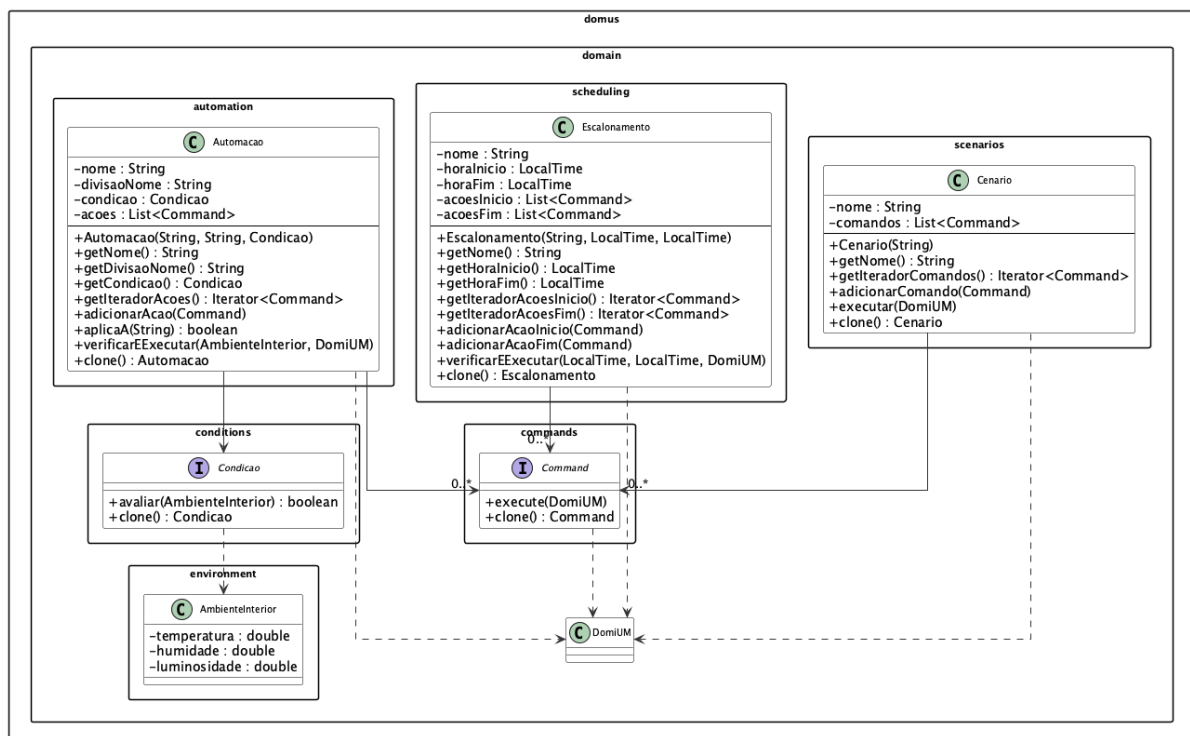


Figura 6. Reutilização de comandos em cenários, automações e escalonamentos.

3.6 Criação extensível de Dispositivos

A criação de dispositivos foi isolada no pacote `domus.domain.factories`. Esta parte da aplicação existe para evitar que a lógica de criação de dispositivos fique espalhada pela fachada, pelos controllers ou pelos gestores.

A interface `DispositivoFactory` define o contrato comum para criar dispositivos. Cada fábrica concreta sabe criar um tipo específico de dispositivo. Por exemplo, `LampadaInteligenteFactory` cria lâmpadas, `ColunaInteligenteFactory` cria colunas, `CortinaInteligenteFactory` cria cortinas, `ArCondicionadoInteligenteFactory` cria aparelhos de ar condicionado, e assim sucessivamente.

A classe `DispositivoRegistry` funciona como um registo de tipos de dispositivos disponíveis. Internamente, associa uma string textual, como `lampada`, `coluna` ou `portao`, à respetiva fábrica. Quando o utilizador indica o tipo de dispositivo na consola, o sistema pede ao registry para criar o dispositivo correto.

Esta solução torna o sistema mais extensível. Para adicionar um novo tipo de dispositivo, basta criar uma nova subclasse de `Dispositivo`, criar a respetiva fábrica e registá-la no `DispositivoRegistry`. A lógica principal da aplicação não precisa conhecer os detalhes de construção desse novo dispositivo.

Este ponto é relevante para o enunciado, porque era pedido que a solução permitisse acrescentar novos tipos de dispositivos através de novas classes, sem obrigar a alterar a classe computacional principal. A classe `DomiUM` não precisa ter um método específico para cada tipo de dispositivo nem conhecer diretamente todas as subclasses concretas.

No fluxo atual, o `controller` apenas recolhe o tipo textual introduzido pelo utilizador. A `DomiUM` valida permissões e delega a criação no `GestorCasas`. O `GestorCasas`, por sua vez, utiliza o `DispositivoRegistry` para obter a fábrica correta. Assim, a criação concreta fica separada das regras de domínio e da interface textual.

A Figura 7 mostra a relação entre a interface `DispositivoFactory`, as fábricas concretas e o `DispositivoRegistry`.



Figura 7. Criação extensível de dispositivos através de factories e registry.

3.7 Histórico e Sugestões automáticas

A aplicação regista interações relevantes dos utilizadores com os dispositivos. Este histórico é usado para gerar sugestões automáticas de escalonamentos, permitindo que o sistema proponha ações recorrentes com base no comportamento observado.

Cada interação é representada pela classe `RegistoInteracao`, no pacote `domus.domain.history`. Um registo guarda a data e hora da interação, o identificador da casa, o identificador do dispositivo e uma descrição da ação realizada. Estes dados são suficientes para perceber que utilizador repetiu uma determinada ação, sobre que dispositivo e em que momento aproximado.

O histórico fica associado ao `Utilizador`. Isto faz sentido porque as sugestões devem depender dos hábitos de cada utilizador, e não apenas do estado global da casa. Assim, dois utilizadores com comportamentos diferentes podem ter sugestões diferentes.

A geração das sugestões é feita pela classe `GeradorSugestoesHistorico`, no pacote `domus.domain.suggestions`. Esta classe analisa o histórico de um utilizador e agrupa interações semelhantes. Para isso, usa uma chave interna composta pela casa, pelo dispositivo, pela ação e pela hora sugerida. Quando uma mesma combinação aparece várias vezes, passa a ser candidata a sugestão.

A sugestão resultante é representada pela classe `SugestaoEscalonamento`. Esta classe guarda o utilizador, a casa, o dispositivo, a ação sugerida, a hora sugerida, o número de ocorrências e uma mensagem explicativa. Desta forma, a interface consegue apresentar ao utilizador uma sugestão legível antes de a aceitar.

A aplicação não cria automaticamente escalonamentos sem confirmação. O utilizador pode listar as sugestões e escolher aceitar uma delas. Quando isso acontece, a classe `SugestaoCommandRegistry` converte a ação textual da sugestão num objeto `Command`. A partir desse comando, a `DomIUM` cria um escalonamento real na casa correspondente.

Esta decisão mantém o utilizador no controlo. O sistema pode propor comportamentos úteis, mas a decisão final continua a pertencer ao utilizador. Isto é importante porque uma sugestão automática pode fazer sentido em alguns contextos, mas não deve alterar a configuração da casa sem autorização.

A Figura 8 mostra a relação entre o histórico de interações, o gerador de sugestões e a criação de comandos a partir de sugestões aceites.

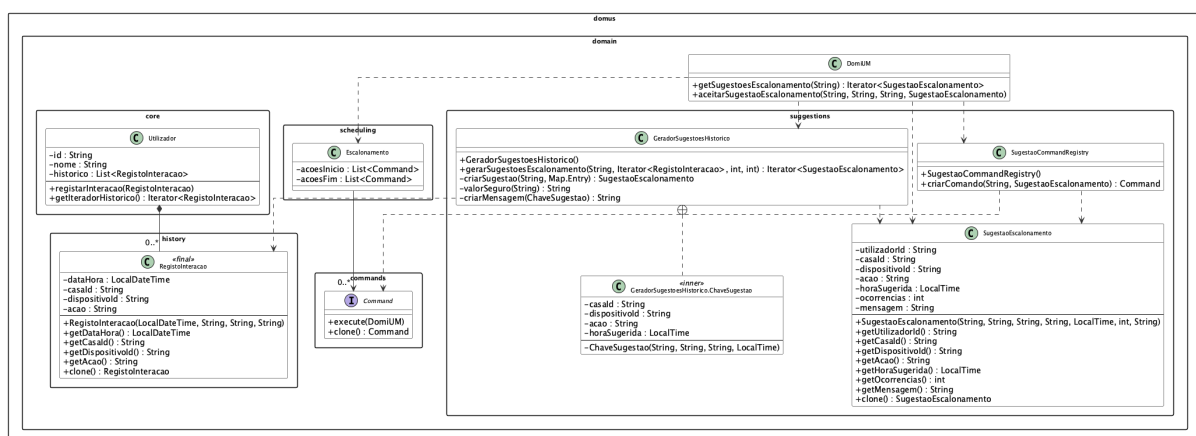


Figura 8. Histórico de interações e geração automática de sugestões.

3.8 Relógio simulado, estatísticas e persistência

Como a aplicação representa um ambiente de simulação, foi necessário criar uma forma de controlar a passagem do tempo. Para isso existe a classe `RelogioSistema`, no pacote `domus.domain.time`. Esta classe guarda a data e hora atuais da simulação e permite avançar o tempo em minutos.

O avanço do tempo é feito através da fachada `DomiUM`. Quando o método de avanço é chamado, a aplicação não altera apenas o relógio. Também acumula tempo nos dispositivos que estão ligados e verifica se algum escalonamento deve ser executado. Esta integração é importante porque os consumos e os escalonamentos dependem diretamente da passagem do tempo.

O consumo dos dispositivos é calculado com base no consumo por hora e no tempo total ligado. Cada dispositivo guarda o seu `consumoPorHora` e o seu `tempoTotalLigado`, em minutos. A fórmula usada é:

```
consumo = consumoPorHora × tempoTotalLigado / 60
```

Desta forma, se um dispositivo consumir 20 Wh e estiver ligado durante 30 minutos, o consumo estimado será 10 Wh. O consumo total de uma casa corresponde à soma dos consumos de todos os dispositivos existentes nas suas divisões.

As estatísticas foram separadas em objetos próprios, no pacote `domus.domain.statistics`. A classe `ResumoCasaConsumo` representa o consumo total de uma casa. A classe `ResumoDispositivoUso` representa dados de utilização de um dispositivo, como consumo atual, tempo total ligado e número de ativações. A classe `ResumoDivisaoDispositivos` representa uma divisão e o número de dispositivos que contém.

Estes objetos são usados apenas para consulta. Em vez de expor diretamente as entidades internas, como `Casa`, `Divisao` ou `Dispositivo`, a aplicação devolve objetos de resumo com a informação necessária para apresentar estatísticas. Esta decisão protege o estado interno do domínio e torna as consultas mais simples.

A aplicação permite obter estatísticas como a casa com maior consumo, os três dispositivos mais usados por tempo ligado, os três dispositivos com mais ativações e as três divisões com mais dispositivos. Também existe um resumo textual do consumo de todas as casas, apresentado com duas casas decimais.

A persistência é feita através de serialização binária de objetos Java. A classe `DomiUM` implementa `Serializable` e contém o estado principal da aplicação, incluindo gestores, casas, utilizadores, dispositivos, cenários, automações, escalonamentos, histórico e relógio simulado. Ao gravar a `DomiUM`, é guardado o estado completo da aplicação.

Foram definidos os métodos `gravarEstado(String)` e `carregarEstado(String)` para guardar e recuperar a aplicação a partir de ficheiro. Em caso de erro, é lançada uma `PersistenciaException`, permitindo que o `controller` apresente uma mensagem clara ao utilizador. Isto evita que a aplicação indique sucesso quando a gravação ou o carregamento falhou.

Esta abordagem cumpre o requisito de guardar e carregar o estado da aplicação em ficheiro binário. Também permite entregar um ficheiro de povoamento final, que pode ser carregado durante a apresentação para demonstrar as funcionalidades sem ter de recriar todos os dados manualmente.

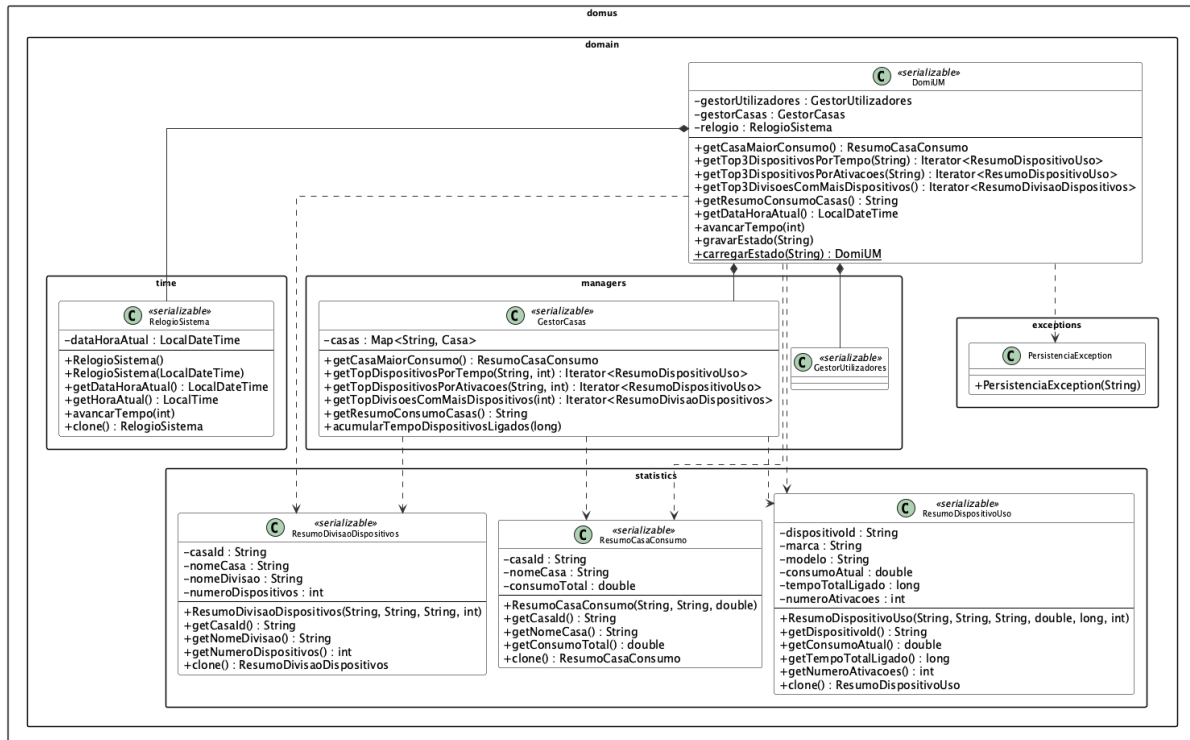


Figura 9. Relógio simulado, estatísticas e persistência.

3.9 Hierarquia de exceções

A aplicação define uma hierarquia própria de exceções no pacote `domus.domain.exceptions`. A raiz desta hierarquia é a classe `DomusException`, que representa uma exceção geral do domínio da aplicação.

A existência de uma exceção base permite que os métodos da fachada `DomiUM` possam lançar erros relacionados com regras de domínio de forma controlada. Ao mesmo tempo, as subclasses tornam possível distinguir diferentes tipos de erro e apresentar mensagens mais específicas ao utilizador.

As exceções foram agrupadas por tipo de problema. A classe `EntidadeJaExisteException` representa situações em que se tenta criar uma entidade que já existe. Dela derivam exceções como `CasaJaExisteException`, `UtilizadorJaExisteException`, `DispositivoJaExisteException`, `DivisaoJaExisteException`, `CenarioJaExisteException`, `AutomacaoJaExisteException` e `EscalonamentoJaExisteException`.

De forma semelhante, a classe `EntidadeNaoEncontradaException` representa situações em que uma entidade necessária para a operação não existe. Dela derivam exceções como `CasaNaoExisteException`, `UtilizadorNaoExisteException`, `DispositivoNaoExisteException`, `DivisaoNaoExisteException`, `CenarioNaoExisteException`, `AutomacaoNaoExisteException` e `EscalonamentoNaoExisteException`.

As violações de regras de negócio são representadas por `OperacaoDominioException`. Um exemplo é `OperacaoInvalidaException`, usada quando uma operação não faz sentido no contexto atual, como tentar aplicar a uma lâmpada um comando específico de coluna. Outro exemplo é `TipoDispositivoInvalidoException`, usada quando o utilizador indica um tipo de dispositivo que não está registado.

As permissões são tratadas através de uma exceção própria, `SemPermissaoException`, que deriva de `PermissaoException`. Esta exceção é lançada quando um utilizador tenta executar uma operação para a qual não tem autorização, por exemplo administrar uma casa onde apenas tem permissão normal.

A persistência também tem uma exceção própria, `PersistenciaException`. Esta exceção é usada quando ocorre uma falha ao guardar ou carregar o estado da aplicação em ficheiro. Desta forma, o `controller` consegue apresentar uma mensagem clara sem substituir o estado atual por um estado vazio ou indicar sucesso quando a operação falhou.

Esta hierarquia permite que os `controllers` tratem erros de forma mais precisa. Por exemplo, um erro de utilizador inexistente pode gerar uma mensagem diferente de um erro de casa inexistente ou de falta de permissão. Isto torna a aplicação mais robusta e melhora a interação com o utilizador.



A Figura 10 apresenta a hierarquia de exceções usada no domínio.

4. Princípios de Programação Orientada a Objetos aplicados

A solução foi desenvolvida com o objetivo de respeitar os princípios fundamentais da Programação Orientada aos Objetos. Estes princípios aparecem em várias partes da aplicação, desde a organização das entidades até à forma como os comandos, dispositivos, exceções e condições foram modelados.

4.1 Encapsulamento

O encapsulamento foi uma preocupação central no desenvolvimento da aplicação. Os atributos das classes estão definidos como `private`, e, sempre que possível como `final`, impedindo que o estado interno dos objetos seja alterado diretamente a partir do exterior.

As coleções internas, como os mapas de casas, divisões, dispositivos, cenários, automações e escalonamentos, não são expostas diretamente. Em vez disso, são disponibilizados métodos específicos ou iteradores sobre cópias protegidas. Esta decisão impede que código externo consiga modificar diretamente o estado interno do domínio sem passar pelas regras da aplicação.

A fachada `DomiUM` também contribui para o encapsulamento. Os `controllers` não acedem diretamente ao `GestorCasas`, ao `GestorUtilizadores` ou aos mapas internos. Todas as operações relevantes passam pela `DomiUM`, que valida permissões, existência de entidades e compatibilidade de comandos antes de delegar a operação.

Um exemplo claro é a execução de comandos sobre dispositivos. O `controller` pode construir um objeto `Command`, mas não altera diretamente o dispositivo. A validação e a execução passam pelo domínio. Assim, a interface consegue representar a intenção do utilizador, mas não consegue alterar livremente o estado interno da aplicação.

4.2 Abstração

A abstração foi usada para esconder detalhes de implementação e permitir que diferentes classes sejam tratadas de forma uniforme. A interface `Command` representa qualquer ação executável sobre o domínio. A interface `Condicao` representa qualquer condição avaliável sobre o ambiente interior. A interface `DispositivoFactory` representa qualquer classe capaz de criar dispositivos.

Estas abstrações permitem escrever código que depende de contratos e não de classes concretas. Por exemplo, um cenário guarda uma lista de `Command`, sem precisar de saber se cada comando liga uma lâmpada, altera o volume de uma coluna ou fecha um portão. Da mesma forma, uma automação avalia uma `Condicao`, sem precisar conhecer o tipo concreto dessa condição.

A classe abstrata `Dispositivo` também é uma abstração importante. Ela representa o que todos os dispositivos têm em comum, como identificador, marca, modelo, consumo por hora, estado ligado ou desligado e tempo total ligado. As subclasses acrescentam apenas o comportamento específico de cada tipo de dispositivo.

4.3 Herança e Polimorfismo

A herança aparece principalmente na hierarquia de dispositivos, na hierarquia de comandos e na hierarquia de exceções.

Na hierarquia de dispositivos, classes como `LampadaInteligente`, `ColunaInteligente`, `CortinaInteligente` e `ArCondicionadoInteligente` herdam de `Dispositivo`. Isto permite que uma divisão guarde dispositivos diferentes numa mesma coleção do tipo `Dispositivo`, tratando-os de forma uniforme quando apenas são necessárias as operações comuns.

O polimorfismo também é usado nos comandos. Todos os comandos implementam a interface `Command`, mas cada classe concreta executa uma ação diferente. Assim, uma lista de comandos pode conter ações variadas e executá-las através do mesmo método `execute()`.

Nas exceções, a herança permite tratar erros de forma geral ou específica. Um `controller` pode capturar uma exceção concreta, como `CasaNaoExisteException`, quando quer mostrar uma mensagem específica, ou capturar `DomusException` como fallback para erros genéricos do domínio.

4.4 Composição

A composição foi usada para representar relações em que uma classe contém objetos que fazem parte do seu estado interno. Por exemplo, uma `Casa` contém divisões, cenários, automações e escalonamentos. Uma `Divisao` contém dispositivos e um `AmbienteInterior`. Um `Utilizador` contém o seu histórico de interações.

A classe `Domium` também é composta por elementos internos fundamentais, como `GestorCasas`, `GestorUtilizadores` e `RelogioSistema`. Estes objetos fazem parte do estado da aplicação e são usados pela fachada para coordenar as operações do domínio.

Esta utilização de composição ajuda a organizar o sistema em objetos com responsabilidades próprias. Em vez de uma única classe guardar todos os dados e todas as regras, cada entidade mantém o estado que lhe pertence.

4.5 Padrões e técnicas de desenho utilizados

Foram aplicados alguns padrões e técnicas de desenho para organizar melhor a solução.

A classe `DomiUM` segue a ideia de uma fachada. Ela oferece um ponto de entrada único para a camada de domínio e esconde os gestores internos da camada de controlo.

O padrão `Command` foi usado para representar ações sobre dispositivos. Cada ação é encapsulada num objeto próprio, que pode ser executado diretamente ou guardado em cenários, automações e escalonamentos.

A criação de dispositivos foi organizada através de `factories`. A interface `DispositivoFactory` define o contrato de criação, e cada fábrica concreta instancia um tipo específico de dispositivo.

O `DispositivoRegistry` funciona como um registo de `factories`. Ele associa nomes textuais a fábricas concretas, permitindo criar dispositivos a partir do tipo introduzido pelo utilizador.

A separação MVC também foi aplicada de forma simples. A `ConsoleView` representa a vista, os controllers coordenam os menus e o pacote `domus.domain` representa o Model, onde ficam as regras de negócio.

Esta combinação de princípios e padrões permitiu construir uma aplicação mais modular, mais fácil de manter e mais preparada para evolução.

5. Funcionalidades da aplicação

A aplicação **Domium** é operada através de menus em modo texto. A navegação é feita por opções numeradas e a interface foi pensada para guiar o utilizador pelas operações disponíveis, apresentando mensagens claras em caso de erro e confirmações quando uma operação é concluída com sucesso.

Ao iniciar, o utilizador seleciona com que conta quer operar. A partir daí, as opções disponíveis dependem das permissões que esse utilizador tem em cada casa. Um administrador tem acesso às operações de configuração, como criar divisões, adicionar dispositivos ou atribuir permissões a outros utilizadores. Um utilizador normal pode interagir com os dispositivos existentes, executar cenários, criar automações e escalonamentos e consultar estatísticas. Esta distinção reflete-se diretamente nos menus, que adaptam as opções ao perfil do utilizador em sessão.

A gestão de dispositivos permite adicionar vários tipos de equipamentos às divisões de uma casa. Cada tipo de dispositivo tem operações próprias acessíveis a partir do menu, como ajustar a intensidade ou a temperatura de cor de uma lâmpada, regular o volume de uma coluna, definir a abertura de uma cortina ou configurar a temperatura alvo de um ar condicionado. Para além das operações específicas, todos os dispositivos podem ser ligados e desligados individualmente.

Os três mecanismos de automação foram construídos para cobrir situações diferentes. Os cenários são executados manualmente e agrupam uma sequência de ações sobre vários dispositivos. O ficheiro de povoamento (criado para testar a aplicação) inclui quatro cenários: Sair de Casa, Jantar com Amigos, Deitar e Acordar. As automações disparam ações de forma automática quando uma condição ambiental é satisfeita numa divisão, como luminosidade abaixo de um limite ou humidade acima de um valor configurado. Os escalonamentos associam ações a horas concretas do dia, executando-as quando o relógio simulado as atinge. O avanço do tempo é feito manualmente pelo utilizador e, além de acionar escalonamentos, atualiza os consumos dos dispositivos que estão ligados.

A aplicação disponibiliza estatísticas sobre a utilização da instalação, nomeadamente a casa com maior consumo total, os três dispositivos mais usados por tempo ligado e por número de ativações, e as três divisões com mais dispositivos. Estas consultas são apresentadas de forma simples e legível na consola.

Com base no histórico de interações de cada utilizador, a aplicação é capaz de sugerir escalonamentos automáticos. Quando o sistema deteta que uma determinada ação foi executada repetidamente à mesma hora, apresenta uma sugestão ao utilizador. Se este a aceitar, o escalonamento é criado diretamente na casa correspondente.

Por fim, o estado completo da aplicação pode ser gravado e carregado a partir de ficheiro binário em qualquer momento. Isto permite retomar uma sessão anterior sem perder dados ou recriar manualmente toda a configuração.

6. Conclusão

O desenvolvimento do `DomiUM` permitiu aplicar de forma concreta os conceitos estudados ao longo da unidade curricular. A aplicação cresceu de forma gradual, começando pela gestão base das entidades e evoluindo até às funcionalidades mais complexas, como as automações, os escalonamentos, as sugestões automáticas e a persistência.

A decisão de organizar o domínio em torno de uma fachada central revelou-se acertada. À medida que novas funcionalidades foram sendo adicionadas, a separação entre interface, controlo e domínio facilitou esse processo sem obrigar a reorganizar o que já estava feito. O padrão Command foi particularmente útil neste aspeto, porque permitiu que cenários, automações e escalonamentos reutilizassem a mesma abstração sem duplicar lógica.

O projeto cumpre todos os níveis de requisitos definidos no enunciado, incluindo a geração automática de sugestões com base no histórico de interações. O ficheiro de povoamento entregue permite demonstrar estas funcionalidades de forma completa durante a apresentação.

No balanço geral, este trabalho foi uma oportunidade para perceber que as decisões de arquitetura tomadas no início têm um impacto direto na facilidade com que a aplicação evolui. Optar por abstrações bem definidas e por responsabilidades claras entre classes tornou o código mais fácil de manter e de estender ao longo do desenvolvimento.

7. Anexos

O diagrama de classes completo da aplicação `Domium` encontra-se no anexo seguinte. Inclui todas as classes, interfaces e relações descritas ao longo do relatório, organizadas pelos pacotes correspondentes.

Nota: para uma melhor visualização, o diagrama encontra-se também no repositório git, juntamente com o código fonte.

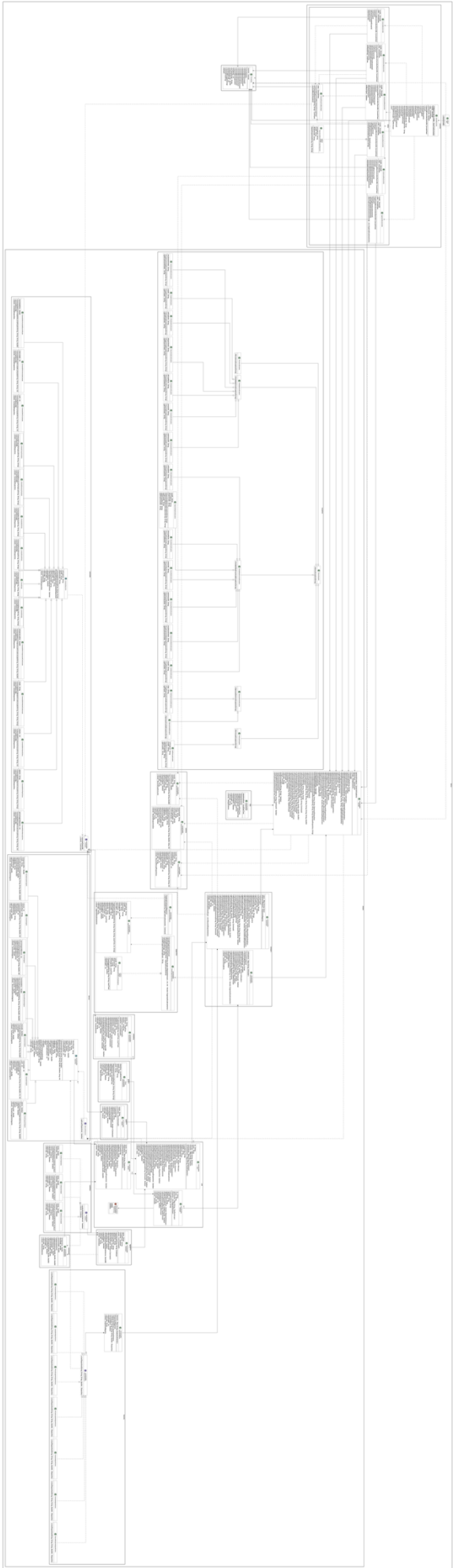


Figura 11. Diagrama de classes completo da aplicação DomiUM.